Nutrition & Health Tracking Platform

Product Requirements Document

Document Version: 1.1
Product Type: Personal-first nutrition, health, medication, and supplement tracking platform
Primary Market: India-first, with architecture suitable for broader use
Implementation Strategy: Phased delivery across Phase 1, Phase 2, and Phase 3

Phase 1 — Core Tracking Platform

1. Phase 1 Objective

Phase 1 establishes the complete foundation of the product.

The application must allow a user to:

- Create an account and securely log in.
- Maintain a personal health and nutrition profile.
- Calculate reference nutrition requirements.
- Review, accept, or manually override nutrition targets.
- Search Indian and general foods using external nutrition providers.
- Review food information before logging.
- Manually override food quantities, states, and nutrition values before confirming a log.
- Log food using quantity and raw/cooked state.
- Track calories and macronutrients.
- Track selected micronutrients.
- Configure which nutrients appear on the dashboard.
- Create custom foods.
- Create and reuse meals.
- Track medications and supplements.
- Support recurring schedules such as daily, alternate-day, weekly, and custom schedules.
- Track taken, missed, skipped, and rescheduled events.
- View historical nutrition and health activity.
- Define a personalized daily reset time.

- Support multiple users concurrently while maintaining strict data isolation.

Phase 1 intentionally does **not** include image recognition, barcode scanning, voice/NLP logging, in-app AI recommendations, MCP, or external AI-agent integration.

---

2. Product Principles

The product must follow these principles throughout Phase 1:

2.1 Tracking over advice

The system is primarily a tracking and information platform.

It should calculate, organize, visualize, and preserve user data.

It must not present itself as a doctor, dietician, pharmacist, or diagnostic system.

2.2 User-controlled targets and logs

The application provides calculated or externally sourced nutrition information, but the user has final control over what is recorded.

The user must be able to review and override values before saving a food log.

2.3 Historical integrity

Historical records must remain stable even when an external provider changes food nutrition data.

2.4 User-specific behavior

Nutrition calculations, tracking days, targets, schedules, and preferences are user-specific.

2.5 API-provider independence

External nutrition providers must be accessed through an abstraction layer so that providers can be replaced without redesigning the application.

2.6 Accuracy over logging speed

The product should prioritize correctness and user verification over automatically logging uncertain information.

3. Authentication and Authorization

3.1 Registration

The system shall support:

- Account creation.
- Email-based authentication.
- Secure password storage.
- Login.
- Logout.
- Session/token management.
- Password reset.

3.2 Roles

The initial authorization model shall support:

`USER`

An optional:

`ADMIN`

role may exist in the data model and authentication layer for future administrative capabilities.

No major admin-specific product feature is required in Phase 1.

3.3 User isolation

Every user-owned entity must be associated with the authenticated user.

Examples include:

- Profile.
- Nutrition targets.
- Food logs.
- Custom foods.
- Meals.
- Health items.
- Health-item events.
- Preferences.
- Historical data.

The backend must enforce ownership.

The frontend must never be treated as the security boundary.

3.4 Security requirements

Passwords must never be stored in plaintext.

Authentication tokens/session data must be handled securely.

Sensitive API credentials must exist only on the backend.

---

4. User Profile

The user profile is the source of personal information required for nutrition calculations and application preferences.

4.1 Profile information

The system should support:

- Name.
- Date of birth or age.
- Sex.
- Height.
- Weight.
- Activity level.
- Goal/preferences needed for calculation methodology.
- Timezone.
- Daily Reset Time.
- Other configurable nutrition preferences.

4.2 Timezone

Timezone must be stored per user.

The application must not assume the server timezone is the user's timezone.

All date-sensitive behavior must use:

```
User Timezone + User Daily Reset Time
```

4.3 Daily Reset Time

Each user can configure the time at which their logical tracking day changes.

Default:

`12:00 AM`

Example:

If a user selects:

`04:00 AM`

their tracking day is:

`04:00 AM → 03:59 AM`

Therefore:

Actual Timestamp	Tracking Day
Sep 24, 11:00 PM	Sep 24
Sep 25, 1:00 AM	Sep 24
Sep 25, 3:59 AM	Sep 24
Sep 25, 4:00 AM	Sep 25

The application must distinguish between:

Daily Reset Time — user-configurable setting.

Tracking Day — logical day used for daily aggregation.

4.4 Tracking-day calculation

Conceptually:

`Timestamp + User Timezone + Daily Reset Time → Tracking Day`

The derived Tracking Day must be used consistently by all daily nutrition aggregation queries.

Changing the reset time must not alter historical timestamps.

Historical events retain their actual timestamps.

4.5 Profile preferences

Profile preferences should include:

- Daily Reset Time.
- Dashboard nutrient selection.
- Units.
- Notification preferences.
- Other product-level settings.

---

5. Nutrition Requirement and Target Engine

The application must distinguish between:

Reference Nutrition Requirement

and

Active User Target

The calculated reference value is not automatically treated as the user's final target.

5.1 Reference target calculation

The system must calculate reference values using:

- User profile information.
- Age/life stage.
- Sex where relevant.
- Height and weight where relevant.
- Activity level where relevant.
- Goal-related inputs where relevant.
- Reference nutrition standards.

Not every nutrient should be calculated using the same formula.

Some nutrients should be sourced from established reference tables rather than calculated directly from body weight.

5.2 Nutrients initially supported

Macronutrients

- Calories.

- Protein.
- Carbohydrates.
- Fat.

Common micronutrients

The underlying model must support additional micronutrients so that Phase 2 can expand coverage without redesigning the core target system.

Examples include:

- Fiber.
- Calcium.
- Iron.
- Magnesium.
- Potassium.
- Sodium.
- Zinc.
- Vitamin A.
- Vitamin C.
- Vitamin D.
- Vitamin B12.
- Folate.

5.3 Reference sources

India-focused reference information should be prioritized.

Possible sources include:

- ICMR/NIN reference values.
- Indian Food Composition Tables where legally and technically appropriate.
- Other recognized nutrition standards.

External reference data must be versioned.

5.4 Active target

The active target is the value currently used by the dashboard and progress calculations.

Example:

Reference protein:

`110 g/day`

User-adjusted active target:

`130 g/day`

The application must preserve both values.

5.5 Target review

After the first calculation or recalculation, the user must be able to review the proposed values.

The UI should clearly indicate:

- Calculated value.
- Source.
- Calculation/reference version.
- Effective date.
- Whether the value is manually overridden.

The user must be able to:

- Accept all.
- Accept individual values.
- Edit values manually.
- Reject a recalculation.

5.6 Manual target overrides

The user may override a target manually.

Example:

`Protein`

Calculated: `110 g`
Active: `130 g`
Status: `Manual Override`

The calculated reference value must still be preserved.

5.7 Target recalculation

The system shall support recalculation when relevant profile information changes.

Example changes:

- Weight.
- Height.
- Activity level.
- Goal.
- Life stage.

Recalculation creates a new proposed target version.

Existing active targets must not silently change.

The user must approve the new values before they become active.

5.8 Target versioning

Every target version should contain, where applicable:

- User ID.
- Nutrient.
- Reference value.
- Active value.
- Unit.
- Source.
- Reference version.
- Calculation version.
- Created timestamp.
- Effective timestamp.
- Override status.
- Override metadata.

---

6. Food Data Provider Architecture

Phase 1 shall use external nutrition providers rather than maintaining a large local master-food database.

The backend must expose an internal provider abstraction.

Conceptually:

`NutritionProvider`

Possible implementations include:

- Provider A.
- Provider B.
- Provider C.

The application should be able to switch providers without changing the rest of the domain model.

6.1 Candidate data sources

Possible sources include:

Open Food Facts

Useful especially for packaged foods and barcode-oriented data.

USDA FoodData Central

Useful as a broad general-purpose nutrition data source.

Indian food/reference sources

Indian-specific datasets should be preferred where appropriate and legally usable.

IFCT/Indian food datasets should not simply be replicated into the application without checking their licensing and permitted electronic use.

The provider abstraction must therefore support replacement of data sources.

6.2 Provider selection

Provider selection should consider:

- Indian food coverage.
- Nutrition completeness.
- API reliability.
- API limits.
- Licensing.
- Cost.
- Query performance.
- Data freshness.

6.3 Provider failure

If a provider is unavailable, the system should return a clear error.

It must not silently substitute invented nutrition values.

The backend must distinguish:

- Provider unavailable.
- Food not found.
- Invalid request.
- Rate-limited request.
- Provider returned incomplete data.

---

7. Food Model

A food is a reusable nutritional item.

A food may originate from:

- External nutrition provider.
- User-created custom food.

The model should retain enough information to distinguish those sources.

Example:

`Jeera Rice`

Attributes may include:

- Food ID.
- User ID where applicable.
- Source provider.
- Provider food ID.
- Name.
- Serving unit.
- Nutrition values.
- Raw/cooked/prepared state.
- Metadata.

---

8. Food Logging

The primary Phase 1 action is:

Log Food

The complete workflow must be:

`Search food → Select food → Configure entry → Review/Confirm → Save`

The system must **not immediately save a food log simply because a food was selected**.

8.1 Food log review and confirmation

Before a food entry is persisted, the application must show a confirmation screen or confirmation state containing the exact information that will be logged.

Example:

```
Confirm Food Log

Jeera Rice
Quantity: 200 g
State: Cooked

Calories: 310 kcal
Protein: 10 g
Carbohydrates: 54 g
Fat: 8 g

[Edit]              [Confirm Log]
```

The purpose is to allow the user to verify the application's interpretation before it becomes historical data.

8.2 User editing before confirmation

The user must be able to change:

- Food item.
- Quantity.
- Unit.
- Raw/cooked/prepared state.
- Meal/category.
- Nutrition values.
- Other supported food-log attributes.

Changing quantity or state should normally trigger recalculation of the nutrition values.

8.3 Per-nutrient manual override

The user must be able to override individual nutrition values before confirming the food log.

Example:

Application estimate:

`Protein = 10 g`

User knows the actual/reference value is:

`Protein = 12 g`

The user can change:

`Protein: 10 g → 12 g`

Similarly, the user may override:

- Calories.
- Protein.
- Carbohydrates.
- Fat.
- Fiber.
- Vitamins.
- Minerals.
- Other available nutrients.

The override must be stored as part of the food-log snapshot.

The user must not be forced to override every nutrient; overrides are optional and field-specific.

8.4 Override semantics

The system should preserve the distinction between:

- Provider/calculated nutrition.
- User-overridden nutrition.
- Final nutrition actually logged.

Example:

```
Protein
Provider value: 10 g
```

```
User override: 12 g
Final logged value: 12 g
```

This provides traceability while allowing the user to control the final record.

8.5 Quantity/state change behavior

Example:

Provider data:

`100 g cooked rice`

User enters:

`200 g cooked`

The system calculates the corresponding nutrition for 200 g.

The user then reviews the result and can override individual values if needed.

8.6 Confirmation requirement

The final `Confirm Log` action should be explicit.

Only after confirmation should the food entry become part of the user's historical record.

8.7 Food logging workflow

The expected workflow is:

```
Dashboard
   ↓
Add Food
   ↓
Search Food
   ↓
Select Food
   ↓
Enter Quantity
   ↓
Select Raw/Cooked/Prepared State
   ↓
Calculate Nutrition
   ↓
Review Nutrition
```

```
   ↓
Optional Manual Overrides
   ↓
Confirm Log
   ↓
Persist Nutrition Snapshot
   ↓
Update Tracking Day
```

---

9. Quantity

Phase 1 must support gram-based quantities.

Example:

`Jeera Rice — 200 g`

Other units may be supported where provider data allows.

The quantity entered by the user must be reflected in the final nutrition calculation.

---

10. Raw/Cooked State

The application must explicitly represent food state.

Examples:

`Rice — 200 g — Cooked`

`Rice — 100 g — Raw`

`Chicken — 150 g — Cooked`

This distinction must be part of the logged food record rather than free-form text.

The purpose is to avoid nutrition ambiguity because raw and cooked values can differ substantially.

---

11. Logged Nutrition Snapshot

When the food is confirmed, the application must persist a nutrition snapshot.

Example:

```
Food:
Jeera Rice

Quantity:
200 g

State:
Cooked

Provider Protein:
10 g

User Override:
12 g

Final Protein:
12 g

Calories:
310 kcal

Carbohydrates:
54 g

Fat:
8 g
```

This snapshot becomes part of the historical record.

If the external provider later changes the food data, historical logs must not automatically change.

---

12. Custom Foods

The user must be able to create a custom food.

A custom food may contain:

- Name.
- Description.
- Serving unit.

- Serving weight.
- Calories.
- Macronutrients.
- Available micronutrients.
- Raw/cooked state where applicable.

Custom foods are owned by the user.

Custom food entries should also follow the same review and confirmation model when they are logged.

---

13. Personal Serving Definitions

Phase 1 should provide basic reusable serving definitions where practical.

Example:

`1 bowl = 250 g`

`1 roti = 45 g`

This allows faster future logging.

---

14. Meals

The application must support combining multiple foods into a meal.

Example:

Breakfast

- 2 eggs.
- 2 rotis.
- 200 ml milk.
- 1 banana.

The meal should have an aggregate nutritional value.

Meals should be reusable.

When a meal is selected for logging, the user should still be shown the final nutrition summary before confirmation.

Individual food items within a meal should remain editable where practical.

---

15. Nutrition Tracking

Phase 1 must support daily tracking of:

- Calories.
- Protein.
- Carbohydrates.
- Fat.

The underlying nutrition model should support additional nutrients.

---

16. Nutrient Configuration

The application should track a broad set of nutrients, but the main dashboard must remain user-configurable.

A user can:

- Enable a nutrient.
- Disable a nutrient.
- Reorder nutrients where supported.
- Restore defaults.

16.1 Default dashboard nutrients

A reasonable default set includes:

- Calories.
- Protein.
- Carbohydrates.
- Fat.
- Fiber.
- Sodium.
- Calcium.
- Iron.
- Potassium.
- Vitamin C.
- Vitamin D.

- Vitamin B12.

The exact default list can be adjusted during implementation.

16.2 Main dashboard

Only nutrients selected by the user should appear prominently on the main dashboard.

Example:

```
Today's Tracking Day
04:00 AM → 03:59 AM

Calories
1,820 / 2,200 kcal

Protein
108 / 130 g

Carbohydrates
210 / 260 g

Fat
58 / 70 g

Fiber
21 / 30 g
```

16.3 Full Nutrition Analysis

The application must provide a separate:

Full Nutrition Analysis

page.

This page displays all nutrients for which data exists.

It must not be limited to the dashboard selection.

---

17. Daily Dashboard

The dashboard represents the user's current Tracking Day.

It should show:

- Current Tracking Day.
- Daily Reset Time.
- Calories.
- Selected nutrients.
- Food log summary.
- Medication/supplement summary.
- Progress toward active targets.

17.1 Tracking Day header

Example:

```
Today
Sep 25

Tracking Day:
04:00 AM – 03:59 AM
```

17.2 Food summary

The dashboard should allow navigation to:

- Breakfast.
- Lunch.
- Dinner.
- Snacks.
- Other meal entries.

Meal naming/classification must remain flexible.

---

18. Medication and Supplement Tracking

Medication and supplements must use the same conceptual feature model.

The domain object should be:

`HealthItem`

with:

`type = MEDICATION | SUPPLEMENT`

This prevents duplicated implementation logic.

18.1 Health item fields

A HealthItem may contain:

- ID.
- User ID.
- Name.
- Type.
- Dosage/quantity as entered by the user.
- Unit.
- Instructions as entered by the user.
- Schedule.
- Start date.
- End date if provided.
- Notes.
- Active/inactive status.

The application stores what the user enters.

It must not independently prescribe or recommend medication.

---

19. Health Item Scheduling

Phase 1 must support:

Daily

Example:

Every day at 8:00 AM.

Alternate day

Example:

Every second day.

Weekly

Example:

Every Monday at 8:00 AM.

Custom

Custom recurrence rules can be represented in a schedule abstraction.

19.1 Schedule events

A scheduled occurrence becomes a health-item event.

Each event may have:

- Scheduled timestamp.
- Status.
- Completion timestamp.
- Notes.

Statuses:

`SCHEDULED`

`TAKEN`

`MISSED`

`SKIPPED`

`RESCHEDULED`

---

20. Missed Medication/Supplement Handling

The system must support visibility into missed events.

Example:

A scheduled supplement event is missed on Monday.

On Tuesday, the user should be able to see that the Monday event was missed.

The application must not tell the user to double a dose or otherwise make a medical decision.

The system is responsible for tracking the event, not determining the correct medical response.

20.1 Carry-forward

Carry-forward behavior may be configurable for eligible tracking items.

The application may surface:

`Missed yesterday`

or:

`Pending event`

but must not reinterpret the event as an instruction to take additional medication.

---

21. Relationship Between Tracking Day and Health Schedules

Daily nutrition tracking uses the user-defined Tracking Day.

Medication/supplement schedules should primarily use actual scheduled timestamps.

This distinction prevents a change in Daily Reset Time from unintentionally changing medication timing.

A schedule explicitly defined in terms of a logical day may use Tracking Day semantics.

---

22. Historical Tracking

The application must support historical views such as:

- Previous Tracking Day.
- Previous week.
- Previous month.
- Custom date range.

Historical nutrition records should show the logical Tracking Day according to the application's date semantics.

Food entries should retain exact timestamps.

---

23. Recent and Favorite Foods

The application should support convenient access to:

- Recently used foods.
- Frequently used foods.
- Favorite foods.

This should reduce repetitive food searching.

---

24. Water Requirement Foundation

Phase 1 may calculate a reference water requirement.

The result must be presented as a reference value rather than an absolute medical requirement.

Water tracking itself is expanded in Phase 2.

---

25. Protein Requirement Foundation

The application must provide a reference protein target using an explainable calculation or reference method.

The active value remains user-editable.

---

26. Micronutrient Requirement Foundation

Phase 1 must establish the target-data architecture for vitamins and minerals.

Values must include:

- Nutrient.
- Amount.
- Unit.
- Population/life-stage context.
- Source.
- Reference/version.

The system must avoid presenting every value as a universally mandatory daily number.

---

27. Reference and Medical Disclaimer

The application must clearly state:

> This application is intended for nutrition, medication, supplement, and health tracking and for reference purposes only. Nutrition values and targets are based on reference data and calculation methodologies and may not reflect individual medical requirements. This application does not provide diagnosis, treatment, prescription, or medical advice.

The disclaimer should appear during relevant onboarding and in appropriate settings/information screens.

---

28. Notifications

Phase 1 notification support should cover health-item schedules.

Possible notifications:

- Scheduled medication/supplement reminder.
- Missed-event notification where configured.

Notification behavior must be user-configurable.

---

29. Phase 1 Screens

The minimum Phase 1 screen set should include:

1. Login.
2. Register.
3. Password reset.
4. Initial setup.
5. Profile.
6. Nutrition target setup.
7. Target review.

8. Dashboard.
9. Food search.
10. Food detail.
11. Log food.
12. Food confirmation/review.
13. Create custom food.
14. Meals.
15. Meal detail.
16. Medication.
17. Supplement/Health Items.
18. Health-item schedule.
19. History.
20. Full Nutrition Analysis.
21. Preferences.
22. Notifications/settings.

---

30. Phase 1 First-Time Setup

Expected journey:

`Register`

→ `Profile`

→ `Nutrition Inputs`

→ `Calculate Reference Targets`

→ `Review Targets`

→ `Accept/Edit`

→ `Configure Dashboard Nutrients`

→ `Configure Daily Reset Time`

→ `Dashboard`

Daily Reset Time defaults to midnight unless changed by the user.

---

31. Phase 1 Food Logging Journey

Expected workflow:

`Dashboard`

→ `Add Food`

→ `Search Food`

→ `Select Food`

→ `Enter Quantity`

→ `Select Food State`

→ `Calculate Nutrition`

→ `Review Food Log`

→ `Edit if Needed`

→ `Optional Nutrient Overrides`

→ `Confirm Log`

→ `Save Nutrition Snapshot`

→ `Dashboard Updated`

Example:

```
Application estimate:
Protein = 10 g

User changes:
Protein = 12 g

Final logged value:
Protein = 12 g
```

The application must retain enough metadata to distinguish the original estimated value from the user's final override.

---

32. Phase 1 Target Refresh Journey

Expected workflow:

`Profile Change`

→ `Recalculate`

→ `Compare Current vs Proposed`

→ `Accept / Edit / Reject`

→ `Create New Target Version`

→ `Update Active Targets`

No silent target changes should occur.

---

33. Data Model — Phase 1

The initial domain model should contain at least:

User

```
id
email
password_hash
role
timezone
created_at
updated_at
```

UserProfile

```
user_id
name
date_of_birth
sex
height
weight
activity_level
goal
```

UserPreferences

```
user_id
daily_reset_time
units
```

notification_preferences
dashboard_nutrient_preferences

NutritionTarget

id
user_id
nutrient
reference_value
active_value
unit
source
reference_version
calculation_version
is_manual_override
override_metadata
effective_at
created_at

Food

id
user_id
source_type
provider
provider_food_id
name
serving_definition
nutrition_data
food_state
created_at
updated_at

FoodLog

id
user_id
food_id
tracking_day
logged_at
quantity
unit
food_state
provider_nutrition_snapshot
user_override_nutrition

```
final_nutrition_snapshot
meal_id
created_at
updated_at
```

The separation between:

`provider_nutrition_snapshot`

`user_override_nutrition`

and

`final_nutrition_snapshot`

is recommended to preserve traceability.

Meal

```
id
user_id
name
food_items
created_at
updated_at
```

HealthItem

```
id
user_id
type
name
dosage
unit
instructions
schedule
start_date
end_date
notes
active
created_at
updated_at
```

HealthItemEvent

```
id
health_item_id
user_id
scheduled_at
status
completed_at
notes
created_at
updated_at
```

`tracking_day` may be derived rather than permanently stored, provided queries consistently apply the user's timezone and reset-time rules. Storing it as a denormalized value is possible for performance, but the derivation rules must remain authoritative.

---

34. API Architecture — Phase 1

The backend should expose domain-oriented APIs.

Examples:

```
/auth/register
/auth/login

/profile
/profile/preferences

/nutrition/requirements
/nutrition/targets
/nutrition/targets/recalculate

/foods/search
/foods/{id}
/foods/custom

/food-logs
/food-logs/preview
/food-logs/confirm
/food-logs/today
/food-logs/history

/meals
```

```
/meals/{id}

/health-items
/health-items/{id}
/health-items/{id}/schedule
/health-items/events

/dashboard
/nutrition/full-analysis
```

A dedicated preview/confirmation flow is recommended for food logging.

---

35. Food Log Preview API

The food logging system should support a non-persistent preview operation.

Example:

```
POST /food-logs/preview
```

Input:

```
food_id
quantity
unit
food_state
optional_nutrition_overrides
```

Response:

```
food details
calculated nutrition
provider nutrition
possible warnings
editable nutrient values
```

The preview must not create a historical log.

The final confirmation operation creates the immutable historical snapshot.

---

36. Daily Aggregation Logic

All "daily" nutrition queries must use:

`User Timezone`

and:

`Daily Reset Time`

The backend should calculate the current Tracking Day before querying aggregate data.

Conceptually:

```
current timestamp
        ↓
convert to user timezone
        ↓
apply daily reset boundary
        ↓
derive tracking day
        ↓
query food logs
        ↓
aggregate final nutrition snapshots
        ↓
compare against active targets
```

This logic must live in a shared backend service rather than being independently implemented by each endpoint.

---

37. Historical Nutrition Integrity

For each food log, the application must preserve:

- Quantity.
- Food state.
- Provider nutrition snapshot.
- User override values, if any.
- Final nutrition snapshot.
- Logging timestamp.
- User.
- Source metadata where applicable.

A provider update must not mutate previously logged nutritional values.

A user override must not modify the original provider record; it modifies the final logged snapshot.

---

38. Offline Behavior

Phase 1 should provide graceful behavior under network failure.

At minimum:

- Previously loaded application data should remain visible where cached.
- Unsaved food logging should not silently disappear.
- Provider failure should be clearly communicated.
- A failed food confirmation must never result in a partially saved record.

Fully offline synchronization can be expanded later.

---

39. Performance and Scalability

Phase 1 must be designed for approximately 5–10 concurrent users initially.

The architecture should remain suitable for growth beyond that.

Requirements:

- Stateless API where practical.
- User-scoped database queries.
- Indexed user-owned data.
- Efficient daily nutrition aggregation.
- Provider calls isolated behind a service abstraction.
- No dependency on an in-memory singleton for user state.

---

40. API Credentials

External provider API keys must remain backend-side.

They must not be embedded in:

- Frontend source code.
- Browser local storage.
- Public Git repositories.

- Client-side JavaScript configuration.

---

41. Phase 1 Engineering Requirements

Engineering should treat the following as foundational modules:

```
Authentication
User Management
Profile
Preferences
Tracking Day Service
Nutrition Target Engine
Nutrition Provider Abstraction
Food Service
Food Preview/Confirmation Service
Food Logging Service
Meal Service
Health Item Service
Notification/Scheduling Service
Dashboard/Aggregation Service
History Service
```

---

42. Phase 1 Definition of Done

Phase 1 is complete when a user can:

1. Register and log in.
2. Complete a profile.
3. Set their timezone and Daily Reset Time.
4. Calculate nutrition reference values.
5. Review and accept targets.
6. Manually override targets.
7. Recalculate targets after profile changes.
8. Search foods using external providers.
9. Distinguish raw/cooked/prepared states.
10. Enter food quantity.
11. Preview the nutrition that will be logged.
12. Review the complete food entry before saving.
13. Edit food quantity/state/details before confirmation.

14. Override individual nutrition values such as protein, calories, carbohydrates, fat, vitamins, or minerals.
15. Confirm the food log explicitly.
16. Preserve provider, override, and final nutrition snapshots.
17. Create custom foods.
18. Create meals.
19. Track calories and macros.
20. Configure dashboard nutrients.
21. View complete available nutrition data.
22. Create medication and supplement records.
23. Configure recurring schedules.
24. Record taken/missed/skipped/rescheduled events.
25. View historical data.
26. Correctly aggregate data according to the user's Tracking Day.
27. Maintain complete user-level data isolation.

Phase 2 — Expanded Nutrition, Health Tracking, and Analytics

1. Phase 2 Objective

Phase 2 expands the core tracker into a substantially richer nutrition and health management platform.

Phase 2 builds directly on the Phase 1 domain model rather than replacing it.

---

2. Comprehensive Micronutrient Tracking

Phase 2 should expand nutrition coverage significantly.

The system should support a comprehensive nutrient schema covering available:

- Vitamins.
- Minerals.
- Macronutrients.
- Fat subtypes.
- Amino-acid-related information where provider data exists.
- Fiber subtypes where appropriate.
- Other available nutrition attributes.

The application must gracefully handle nutrients that are unavailable for a particular food.

Missing provider data must not be displayed as zero unless zero is actually known.

---

3. Full Nutrition Analysis

The Full Nutrition Analysis page becomes a comprehensive nutrition report.

For each nutrient, display where available:

- Consumed.
- Target/reference.
- Unit.
- Percentage of active target.
- Contributing foods.
- Daily history.

Users should be able to expand/collapse nutrient groups.

Suggested groups:

- Energy.
- Macronutrients.
- Fat.
- Fiber.
- Vitamins.
- Minerals.
- Other nutrients.

---

4. Nutrition Detail Drill-Down

Users should be able to select a nutrient and see:

- Daily intake.
- Target.
- Historical intake.
- Foods contributing most to intake.
- Meal-level contribution.

Example:

`Protein`

→ Daily total

→ Breakfast contribution

→ Lunch contribution

→ Dinner contribution

→ Individual food contribution.

---

5. Recipes

Phase 2 introduces reusable recipes.

A recipe consists of:

- Ingredients.
- Ingredient quantities.
- Ingredient states.
- Number of servings.
- Preparation information.
- Calculated nutritional totals.

5.1 Recipe precision

The system should calculate recipe nutrition from ingredient quantities.

Example:

```
Chicken Biryani


Rice: 300 g raw
Chicken: 500 g raw
Oil: 30 g
Onion: 150 g
Spices: ...
Final servings: 4
```

The system calculates the recipe's nutrition and can divide the resulting total across servings.

---

6. Raw/Cooked Recipe Handling

Recipe ingredients must preserve food state where applicable.

The model must distinguish:

`Raw`

`Cooked`

`Prepared`

This prevents accidental mixing of incompatible nutrition values.

---

7. Barcode Scanning

Phase 2 should add barcode scanning for packaged foods.

Workflow:

`Scan Barcode`

→ `Find Product`

→ `Review Nutrition`

→ `Log`

If a product is not found:

`Product Not Found`

The system may allow manual product creation.

Barcode functionality should use an external provider abstraction rather than hard-coding a single provider.

---

8. Expanded Indian Food Support

Phase 2 should improve Indian-food coverage.

Priority areas may include:

- Indian regional foods.

- Home-style Indian meals.
- Common packaged foods in India.
- Indian restaurant-style foods.
- Frequently used household ingredients.

The system should continue to respect data licensing and provider restrictions.

---

9. Water Tracking

Phase 2 adds explicit water logging.

Users can log:

- Amount.
- Unit.
- Time.

Example:

```
500 ml water
```

Water consumption should be associated with the user's Tracking Day.

The dashboard should show:

```
Water consumed / Water reference target
```

The product should continue to present water requirements as reference values rather than medical mandates.

---

10. Weight Tracking

Users should be able to record weight historically.

Each record may contain:

- Weight.
- Unit.
- Timestamp.
- Tracking Day.
- Optional note.

The application should display trends over time.

---

11. Activity Tracking

Phase 2 should support basic activity logging.

Possible fields:

- Activity type.
- Duration.
- Intensity.
- Timestamp.
- Notes.

Activity can be used as an input to nutrition calculations where appropriate, but the application should clearly distinguish estimated expenditure from measured values.

---

12. Advanced Medication and Supplement Scheduling

Phase 2 expands HealthItem scheduling.

Possible capabilities include:

- More complex recurrence patterns.
- Multiple scheduled times per day.
- Date ranges.
- Specific weekdays.
- Specific intervals.
- Pause/resume.
- Schedule modifications.
- Historical schedule changes.

Every generated event must remain auditable.

---

13. Schedule Versioning

When a medication/supplement schedule changes, the system should not rewrite historical events.

Example:

Original:

`8:00 AM`

Later:

`9:00 AM`

Historical events remain associated with the original schedule.

New events use the new schedule.

---

14. Analytics

Phase 2 adds historical analytics.

Examples:

Nutrition

- Calories over time.
- Protein over time.
- Macro distribution.
- Nutrient adequacy trends.
- Meal contribution trends.

Weight

- Weight trend.
- Change over selected period.

Water

- Daily intake trend.

Health Items

- Adherence history.
- Missed-event frequency.
- Taken vs scheduled events.

Analytics must be descriptive rather than diagnostic.

15. Saved Meals

Phase 2 should make repeated meals easier to log.

Users can save:

`Breakfast`

`Post Workout Meal`

`Lunch`

`Dinner`

or any custom name.

A saved meal can be logged as a group and then adjusted before final submission.

The final saved meal log must continue to use the Phase 1 confirmation and snapshot principles.

---

16. Personalized Serving Definitions

Phase 2 should support richer personal serving units.

Examples:

`1 katori = 150 g`

`1 bowl = 250 g`

`1 glass = 250 ml`

`1 roti = 40 g`

`1 piece = 80 g`

The system should maintain user-specific mappings.

The original nutritional values must remain traceable.

---

17. Meal and Recipe Recalculation

If a reusable meal or recipe changes, historical food logs must remain unchanged.

Future uses should use the latest saved definition.

A logged meal should therefore contain a snapshot of the actual nutritional composition used during logging.

---

18. Dashboard Expansion

The Phase 2 dashboard can include:

- Calories.
- User-selected nutrients.
- Water.
- Weight.
- Activity.
- Health-item adherence.
- Historical trend entry points.

The user's nutrient selection remains respected.

The Full Nutrition Analysis remains the comprehensive view.

---

19. History Expansion

Historical views should support:

- Tracking-day selection.
- Week.
- Month.
- Custom date range.
- Nutrition trends.
- Weight trends.
- Water trends.
- Health-item events.

The UI should make it clear that the application uses a user's configured Tracking Day.

---

20. Phase 2 Notifications

Expanded notifications can include:

- Health-item reminders.
- Missed-event reminders.
- Configurable water reminders.
- Other user-created tracking reminders.

Notifications must remain configurable and must not become medical recommendations.

---

21. Phase 2 Data Model Additions

Potential entities include:

Recipe

```
id
user_id
name
ingredients
servings
nutrition
created_at
updated_at
```

RecipeIngredient

```
food_id
quantity
unit
food_state
```

WaterLog

```
id
user_id
quantity
unit
logged_at
tracking_day
```

WeightLog

```
id
user_id
weight
unit
logged_at
tracking_day
```

ActivityLog

```
id
user_id
activity_type
duration
intensity
logged_at
tracking_day
metadata
```

SavedMeal

```
id
user_id
name
food_items
nutrition_snapshot
created_at
updated_at
```

---

22. Phase 2 Engineering Considerations

The architecture should introduce shared services for:

```
Recipe Calculation
Advanced Scheduling
Analytics
Water Tracking
Weight Tracking
Activity Tracking
Historical Aggregation
Barcode Provider
Serving Definitions
```

These modules must continue using the Phase 1 user-isolation and Tracking Day infrastructure.

---

23. Phase 2 Definition of Done

Phase 2 is complete when:

- Comprehensive micronutrient data is supported.
- Full Nutrition Analysis provides detailed nutrient visibility.
- Recipes calculate nutrition from ingredients.
- Raw/cooked recipe states are preserved.
- Barcode scanning works for supported products.
- Indian food coverage is expanded.
- Water can be tracked.
- Weight history can be tracked.
- Activity can be tracked.
- Advanced health-item schedules are supported.
- Schedule changes preserve historical events.
- Nutrition, weight, water, and adherence analytics are available.
- Saved meals are reusable.
- Personalized serving definitions are supported.
- All Phase 1 capabilities remain functional, including food review and per-nutrient overrides.

Phase 3 — AI, Automation, External Agents, MCP, and Data Export

1. Phase 3 Objective

Phase 3 introduces intelligent input methods and external system integration.

The objective is to minimize manual data entry while preserving user control and data accuracy.

Phase 3 includes:

- Nutrition-label image recognition.
- Food image recognition.
- Natural-language food logging.

- Voice-based logging.
- External AI-agent integration.
- MCP.
- Scoped agent permissions.
- Audit logs.
- CSV/JSON exports.

Phase 3 does **not** require an in-app AI nutrition recommendation engine.

---

2. Nutrition Label Image Recognition

Users can photograph a nutrition label.

Example:

```
Serving Size: 100 g

Calories: 420 kcal
Protein: 12 g
Carbohydrates: 65 g
Fat: 14 g
Sodium: 400 mg
```

The system extracts the information.

2.1 Label processing workflow

```
Upload Image
      ↓
Image Processing
      ↓
Nutrition Label Recognition
      ↓
Structured Nutrition Data
      ↓
User Review
      ↓
Confirm
      ↓
Save / Log
```

---

3. Mandatory User Confirmation

Recognized nutrition information must not automatically become trusted historical data.

The system must show the extracted result for user review.

The user can:

- Accept.
- Edit.
- Reject.

Only confirmed data should be saved as the user's nutrition record.

Per-nutrient manual overrides must also be supported here using the same override model established in Phase 1.

---

4. Food Image Recognition

Users can upload or capture a food image.

Example:

A photograph contains:

- Rice.
- Dal.
- Sabzi.
- Curd.

The system may identify probable food items.

The output must be treated as an estimate.

The user must be able to review:

- Identified foods.
- Quantities.
- Preparation state.
- Nutrition values.

The user must be able to edit or override individual values before confirmation.

Only after confirmation should the food be logged.

---

5. Natural-Language Food Logging

The application should accept input such as:

> "I had two rotis, one bowl dal, 150 grams cooked rice and a glass of milk."

The system converts this into structured food entries.

Conceptually:

```
Natural Language
        ↓
Entity Extraction
        ↓
Food Matching
        ↓
Quantity Extraction
        ↓
Raw/Cooked State Detection
        ↓
Nutrition Lookup
        ↓
User Review
        ↓
User Edits / Overrides
        ↓
Confirm
        ↓
Log
```

---

6. Ambiguity Handling

Natural-language input may be ambiguous.

Example:

> "I had a bowl of rice."

A bowl size may be unknown.

The system should ask for clarification or use a clearly identified user-defined serving definition when one exists.

It must not present an unsupported quantity as certain.

---

7. Voice Logging

The same natural-language processing functionality should be available through voice.

Example:

> "I ate 200 grams cooked chicken and two rotis."

The application converts speech to text and then processes the text through the same structured logging pipeline.

Voice should therefore not create a separate nutrition logic layer.

---

8. AI Logging Accuracy Principle

AI-assisted logging must optimize for:

Accuracy + User Review + Traceability

Where confidence is low, the system should request confirmation.

The system should avoid silently guessing:

- Food identity.
- Quantity.
- Raw/cooked state.
- Serving size.
- Nutrition values.

---

9. External AI Agents

Phase 3 must allow external agents/assistants to access application functionality.

Examples could include:

- Google Assistant-like assistants.
- Future AI assistants.
- Custom AI agents.
- Personal automation agents.

The external agent should interact with the application's APIs/services through controlled interfaces.

---

10. MCP

MCP is a Phase 3 feature.

MCP is intended primarily to enable external AI agents and assistants to interact with the application.

MCP must not be required by the core Phase 1 or Phase 2 application.

---

11. MCP Architecture

The architecture must be:

```
External AI Agent
        ↓
       MCP
        ↓
Application MCP Adapter
        ↓
Application Services
        ↓
Database / External Providers
```

The external agent must **not** receive direct database access.

MCP should expose controlled application-level operations.

---

12. Candidate MCP Tools

Phase 3 should support tools such as:

```
search_food
get_food_nutrition
log_food
get_daily_nutrition
get_nutrition_history
get_health_items
get_health_item_schedule
mark_health_item_taken
get_missed_health_items
get_goals
get_profile
export_nutrition_history
```

The actual tool names may be changed during engineering.

---

13. MCP Read Permissions

Read-only operations should include things such as:

- Current nutrition.
- Nutrition history.
- Profile information.
- Active goals/targets.
- Health-item schedules.
- Logged health-item events.

Read permissions must be scoped.

An agent should not automatically receive all user data.

---

14. MCP Write Permissions

Write operations may include:

- Log food.
- Mark health item taken.
- Create supported tracking records.
- Trigger exports.

Write capabilities require explicit authorization.

Food logging through MCP must still go through the same application validation and confirmation/authorization rules established by the core system.

---

15. Agent Permission Model

The permission model should distinguish between:

`READ`

`WRITE`

`EXPORT`

Potential scopes may include:

```
nutrition.read
nutrition.write
history.read
health.read
health.write
profile.read
export.create
```

Permissions should be granted by the user.

The default should be minimum required access.

---

16. Agent Security

External agents must not directly access:

- Database credentials.
- Internal service credentials.
- Nutrition provider API keys.
- Other users' data.

Every external request must be authenticated and authorized.

---

17. Agent Audit Logging

All agent activity should be auditable.

An audit record may contain:

```
actor_type
agent_id
user_id
action
resource
timestamp
result
permission_scope
request_id
```

Sensitive payloads should not be logged unnecessarily.

---

18. MCP Data Isolation

MCP must obey the exact same authorization boundaries as the main application.

An agent authenticated for User A must never retrieve User B's:

- Nutrition.
- Health items.
- Targets.
- Profile.
- History.

---

19. Export

Phase 3 must support exporting user data.

At minimum:

CSV

Suitable for spreadsheets and analysis.

JSON

Suitable for developers, automation, and AI agents.

---

20. Export Service

Export must be implemented as an application service.

Conceptually:

```
Web UI
   ↓
Export Service
   ↓
Data Serialization
   ↓
CSV / JSON
```

and:

```
MCP
 ↓
Export Service
 ↓
CSV / JSON
```

There should be one authoritative export service rather than separate export logic for the UI and MCP.

---

21. Exportable Data

Users should be able to export:

- Food logs.
- Nutrition snapshots.
- Meals.
- Recipes.
- Nutrition targets.
- Weight history.
- Water history.
- Activity history.
- Medication/supplement records.
- Health-item events.
- Relevant profile settings where appropriate.

---

22. Export Formats

Example JSON structure:

```
{
  "user": {},
  "nutrition_targets": [],
  "food_logs": [],
  "meals": [],
  "recipes": [],
  "water_logs": [],
  "weight_logs": [],
  "activity_logs": [],
  "health_items": [],
  "health_item_events": []
}
```

CSV exports may use separate files or logically separated datasets.

Food-log exports should preserve provider values, user overrides, and final logged values where applicable.

---

23. AI and Medical Safety Boundary

Phase 3 must preserve the product boundary.

The AI layer may:

- Interpret user input.
- Extract structured information.
- Match foods.
- Estimate quantities.
- Explain recorded data.
- Retrieve user history where authorized.

The system must not independently:

- Diagnose conditions.
- Prescribe treatment.
- Change medication dosage.
- Recommend medication changes.
- Present medical decisions as certain.

24. No In-App AI Recommendation Engine

Phase 3 does not require a chatbot that tells users:

- What to eat.
- Which medication to take.
- Which supplement to start.
- Which treatment to follow.

External agents may analyze exported or authorized data, but that is separate from the core tracking product.

---

25. Phase 3 Data Model Additions

Potential entities include:

Agent

```
id
name
provider
status
created_at
```

AgentPermission

```
agent_id
user_id
scope
granted_at
revoked_at
```

AgentAuditLog

```
id
agent_id
user_id
action
resource
timestamp
result
```

```
request_id
metadata
```

ExportJob

```
id
user_id
requested_by
format
scope
status
created_at
completed_at
file_reference
```

---

26. Phase 3 AI Processing Architecture

AI processing should be isolated from core domain services.

Recommended architecture:

```
AI Input Layer
   ↓
AI/NLP Service
   ↓
Structured Logging Intent
   ↓
Application Validation
   ↓
Nutrition Provider / Existing Food Service
   ↓
User Review + Overrides
   ↓
Core Logging Service
```

The AI layer must never bypass the core validation rules.

---

27. Phase 3 Confirmation Model

For AI-generated records, the application should maintain the concept:

`PROPOSED`

followed by:

`CONFIRMED`

or:

`REJECTED`

The proposal must allow user edits and per-nutrient overrides before confirmation.

This is especially important for:

- Food image recognition.
- Nutrition-label extraction.
- Natural-language logging.
- Voice logging.

---

28. Phase 3 Definition of Done

Phase 3 is complete when:

- Nutrition labels can be extracted from images.
- Extracted labels require user confirmation.
- Extracted values can be manually edited or overridden.
- Food images can produce proposed food entries.
- Food-image results can be edited or overridden before logging.
- Natural-language food logging works.
- Voice logging works using the same structured logging engine.
- External agents can interact with the application through controlled interfaces.
- MCP is available for authorized external agents.
- MCP has scoped permissions.
- MCP operations are audited.
- User data remains isolated.
- CSV export works.
- JSON export works.
- Export functionality is available both through normal application services and authorized external-agent access.
- AI processing cannot bypass core validation or authorization.
- Phase 1 and Phase 2 capabilities remain functional.

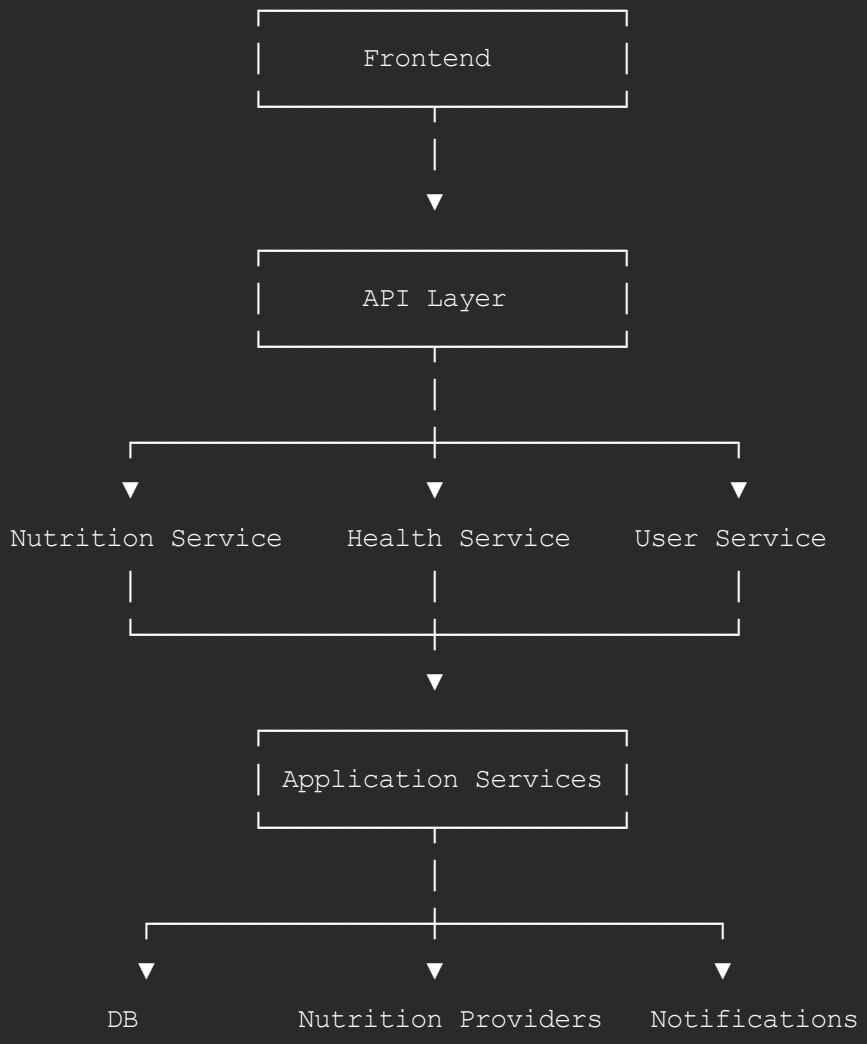
Cross-Phase Engineering Rules

The following rules apply throughout all three phases.

1. Architecture

The system should be organized around clear domain boundaries.

Recommended high-level architecture:



Phase 3 adds:

```
External Agents
      ↓
     MCP
      ↓
Application Services
```

and:

```
AI Processing
      ↓
```

```
Core Application Services
```

---

2. Core Domain Services

The product should ultimately have clear ownership for:

```
Identity Service
Profile Service
Preference Service
Tracking Day Service
Nutrition Requirement Service
Nutrition Target Service
Food Service
Food Provider Service
Food Preview Service
Food Logging Service
Meal Service
Recipe Service
Health Item Service
Schedule Service
Notification Service
Analytics Service
Export Service
AI/NLP Service
Agent/MCP Service
Audit Service
```

Not every service needs to be a separately deployed microservice.

The initial implementation may use modular services within one deployable backend, provided domain boundaries are explicit.

---

3. Tracking Day as a First-Class Concept

Tracking Day is one of the most important domain concepts in the system.

It must be used consistently for nutrition-related daily aggregation.

The system should never scatter custom date calculations throughout the codebase.

A shared function/service should conceptually provide:

```
getTrackingDay(timestamp, userTimezone, dailyResetTime)
```

Every daily nutrition query should depend on this shared behavior.

---

4. Immutable Historical Records

Historical events must not be silently rewritten because current definitions changed.

Examples:

If a food's provider data changes:

Old food logs remain unchanged.

If a recipe changes:

Old logs remain unchanged.

If a target changes:

Historical target versions remain available.

If a medication schedule changes:

Historical events remain associated with the historical schedule.

If a user manually overrides food nutrition:

The override applies to that particular logged record and does not mutate the external provider dataset.

---

5. User Data Isolation

Every data-access path must enforce:

`Authenticated User → Authorized User-Owned Resource`

This applies to:

- REST APIs.
- Internal services.
- Background jobs.
- Notifications.
- MCP.

- Exports.
- Analytics.
- AI-generated actions.

---

6. External Provider Abstraction

The application must never make the core domain dependent on a single nutrition provider.

The provider interface should support operations such as:

```
searchFood()
getFood()
getNutrition()
searchPackagedFood()
lookupBarcode()
```

The exact interface evolves between phases.

---

7. Missing Nutrition Data

When a provider does not supply a nutrient:

The system should represent the value as:

`UNKNOWN / NOT AVAILABLE`

rather than:

`0`

unless zero is a verified value.

This is important for accurate nutrition calculations.

---

8. Calculation and Override Transparency

Where the application calculates or retrieves a value, it should be possible to identify:

- Original/provider value.
- User override, if any.

- Final value used for the record.
- Calculation method where applicable.
- Reference source.
- Calculation version.
- Timestamp.

This applies especially to:

- Nutrition targets.
- Food logs.
- AI-generated nutrition.
- Nutrition-label extraction.

---

9. Correctness over Speed

The product should prefer:

`Accurate + reviewable`

over:

`Fast + silently guessed`

Any uncertain food-log interpretation should be reviewable before persistence.

---

10. Privacy

The application should minimize collection of unnecessary personal information.

User data should be accessible only to:

- The authenticated user.
- Authorized application services.
- Explicitly authorized external agents.

External agents should receive only the minimum information needed for the requested operation.

---

11. Observability

Production implementations should provide:

- Structured application logs.
- Error tracking.
- Request correlation IDs.
- Provider error metrics.
- Authentication failure monitoring.
- Job failure monitoring.
- MCP audit logs from Phase 3.

The system should avoid logging unnecessary sensitive personal information.

---

12. Database Indexing

Indexes should be created around frequent access patterns.

Examples:

```
user_id
user_id + logged_at
user_id + tracking_day
health_item_id + scheduled_at
user_id + nutrient
user_id + created_at
```

Indexes should be validated against actual query patterns.

---

13. API Versioning

Public/external APIs should use an explicit version strategy.

Example:

```
/api/v1/...
```

MCP tool definitions should also be versionable so that external agents are not unexpectedly broken by internal API changes.

---

14. Background Jobs

Background processing may eventually handle:

- Notification generation.
- Recurring health-item events.
- Analytics aggregation.
- Export generation.
- AI image processing.
- AI transcription.
- Provider synchronization where needed.

Background jobs must remain user-scoped.

---

15. Testing Strategy

Testing should cover:

Unit tests

- Nutrition calculations.
- Tracking Day calculation.
- Target versioning.
- Food-state handling.
- Food preview calculations.
- Nutrition override handling.
- Food confirmation behavior.
- Health-item recurrence.
- Nutrition aggregation.
- Permissions.

Integration tests

- Food provider integration.
- Authentication.
- Database operations.
- Scheduling.
- Export generation.
- Food preview and confirmation APIs.

End-to-end tests

Core user journeys should include:

`Register → Setup → Target Review → Search Food → Review → Override → Confirm → Dashboard`

and:

`Create Health Item → Schedule → Event → Mark Taken/Missed`

Phase 3 should additionally cover:

`AI Input → Proposed Record → Edit/Override → Confirmation → Logging`

and:

`Agent → MCP → Permission Check → Application Service`

---

16. Nonfunctional Requirements

The application should target:

Reliability

No loss of confirmed historical records.

Security

Server-enforced authorization and secure credential management.

Scalability

Architecture capable of moving beyond the initial 5–10-user usage profile.

Maintainability

Clear module boundaries and provider abstraction.

Performance

Daily dashboard retrieval should remain efficient even as food-log history grows.

Consistency

All date-sensitive functionality must use the user timezone and applicable Tracking Day semantics.

Auditability

Values that originate from external providers or user overrides should remain traceable.

---

17. Explicitly Out of Scope

The product does **not** include:

- Food inventory management.
- Grocery inventory.
- Food expiry tracking.
- Stock tracking.
- Automatic food purchasing.
- Food recommendations in the core dashboard.
- Medication prescribing.
- Medication-dose recommendations.
- Medical diagnosis.
- Treatment recommendations.
- An in-app medical chatbot.
- Silent AI-based health decisions.
- Direct database access for external AI agents.

---

18. Final Phase Boundaries

Phase 1 — Core Tracking Foundation

```
Authentication
Profile
Preferences
Daily Reset Time
Tracking Day
Nutrition requirements
Nutrition targets
Target versioning
Food search
External nutrition providers
Food preview
Food confirmation
Food quantity editing
Raw/cooked state
Calories/macros
```

```
Selected micronutrients
Per-nutrient food overrides
Custom foods
Meals
Medication tracking
Supplement tracking
Scheduling
Missed events
Dashboard
History
```

Phase 2 — Advanced Nutrition and Health Tracking

```
Comprehensive micronutrients
Full Nutrition Analysis
Recipes
Recipe nutrition calculation
Barcode scanning
Expanded Indian foods
Water tracking
Weight tracking
Activity tracking
Advanced scheduling
Analytics
Saved meals
Personal serving definitions
```

Phase 3 — AI and External Automation

```
Nutrition-label image recognition
Food image recognition
Natural-language logging
Voice logging
AI-assisted structured entry
External agents
MCP
Scoped agent permissions
Agent audit logs
CSV export
JSON export
External data access/automation
```

---

19. Product Definition

At completion of all three phases, the product will be a personal-first nutrition and health tracking platform capable of:

```
Understand User
      ↓
Calculate Reference Requirements
      ↓
Let User Define Active Targets
      ↓
Find Food
      ↓
Estimate Nutrition
      ↓
Let User Review / Edit / Override
      ↓
Confirm Food Log
      ↓
Track Nutrition
      ↓
Track Water / Weight / Activity
      ↓
Track Medication / Supplements
      ↓
Preserve Historical Data
      ↓
Analyze History
      ↓
Accept AI-Assisted Input
      ↓
Expose Authorized Data to External Agents
      ↓
Export User Data
```

The core philosophy remains:

The application records and organizes the user's health and nutrition data, provides transparent reference calculations, and gives the user control over what is ultimately recorded.

---

20. Recommended Engineering Order

Phase 1 dependency sequence

```
Authentication
        ↓
User/Profile
        ↓
Preferences + Timezone
        ↓
Tracking Day Service
        ↓
Nutrition Reference/Target Engine
        ↓
Nutrition Provider Abstraction
        ↓
Food Model
        ↓
Food Search
        ↓
Food Preview
        ↓
Food Review/Edit
        ↓
Nutrition Override
        ↓
Food Confirmation
        ↓
Nutrition Aggregation
        ↓
Dashboard
        ↓
Custom Foods + Meals
        ↓
HealthItem
        ↓
Scheduling/Event Engine
        ↓
History
        ↓
Notifications
```

Phase 2 dependency sequence

```
Phase 1 Nutrition Model
        ↓
Comprehensive Nutrient Schema
        ↓
```

```
Full Nutrition Analysis
        ↓
Recipe Engine
        ↓
Barcode Provider
        ↓
Water / Weight / Activity
        ↓
Advanced Scheduling
        ↓
Analytics
        ↓
Saved Meals / Serving Definitions
```

Phase 3 dependency sequence

```
Existing Food/Logging Services
        ↓
AI Structured-Input Layer
        ↓
Image Recognition
        ↓
Natural Language
        ↓
Voice
        ↓
Agent Authentication
        ↓
Permission System
        ↓
MCP Adapter
        ↓
Audit Logging
        ↓
Export Service
        ↓
MCP Export Integration
```

---

21. Final Engineering Acceptance Criteria

The complete product should satisfy the following foundational rules:

1. Every user has isolated data.

2. Every daily nutrition calculation uses the user's timezone.
3. Every daily nutrition calculation uses the user's configured Daily Reset Time.
4. Tracking Day is a shared domain concept.
5. Every food entry is reviewed before it becomes a confirmed historical log.
6. Users can edit food quantity, unit, food state, and supported entry details before confirmation.
7. Users can override individual nutrition values such as protein, calories, carbohydrates, vitamins, and minerals before confirmation.
8. The system preserves original/provider values, user overrides, and final logged values where applicable.
9. Confirmed historical food nutrition values are immutable.
10. Historical targets are versioned.
11. Manual target changes remain distinguishable from calculated values.
12. External food providers are abstracted behind a provider layer.
13. Missing nutrition data is never silently treated as zero.
14. Raw/cooked/prepared state is explicit wherever nutrition can depend on it.
15. Medication and supplement tracking use a common HealthItem model.
16. Medication/supplement schedules generate trackable events.
17. Missed events are tracked without making medication decisions.
18. Dashboard nutrients are configurable by the user.
19. Full Nutrition Analysis can expose all available nutrient data.
20. Phase 2 capabilities extend Phase 1 rather than replacing it.
21. AI-generated records require validation and user confirmation.
22. AI-generated records support user edits and per-nutrient overrides.
23. External agents never receive direct database access.
24. MCP always uses application authorization rules.
25. Agent actions are auditable.
26. Export is implemented as a reusable application service.
27. The application remains a tracking/information system rather than a medical decision-making system.

Add a concise executive summary Separate requirements from implementat